

LR PARSERS

Terms Used in LR Parser

Item:

→ Shift reduce decisions are made by maintaining states.

States → Set of items

to keep track of where we are in a parse.

LR(0) Items:

→ An LR(0) item of a grammar G is a production of G with a dot at some position of the body.

→ Thus production $A \rightarrow xyz$ yields four items

$$A \rightarrow \cdot xyz$$

$$A \rightarrow x \cdot yz$$

$$A \rightarrow xy \cdot z$$

$$A \rightarrow xyz \cdot$$

→ Production for $A \rightarrow \epsilon$, is $A \rightarrow \cdot$

Canonical LR(0) Collection

→ One collection of set of LR(0) items called the canonical LR(0) collection, provides the basis for constructing a DFA, that is used to make parsing decision.

Augmented Grammar

→ If G is a grammar with start symbol S , then G' , the augmented grammar for G , is G with a new start symbol S' and the production $S' \rightarrow S$.

$$S \rightarrow Aa$$

↓

$$S' \rightarrow S \quad (\text{Augmented grammar})$$

$$S \rightarrow Aa$$

Closure of Item Sets

→ For a grammar G , I is a set of item and $CLOSURE(I)$ can be constructed by two rules

1. Add every item in I to $CLOSURE(I)$.
2. If $A \rightarrow \alpha \cdot B \beta$ is in $CLOSURE(I)$ and $B \rightarrow \gamma$ is production, then add item $B \rightarrow \cdot \gamma$ to $CLOSURE(I)$

$$E \rightarrow E + T / T$$

$$T \rightarrow T * F / F$$

$$F \rightarrow (E) / id$$

Augmented Grammar $\Rightarrow$

$$E' \rightarrow E$$

$$E \rightarrow E + T / T$$

$$T \rightarrow T * F / F$$

$$F \rightarrow (E) / id$$

→ If I is the set of one item $\{[E' \rightarrow \cdot E]\}$,
 then $CLOSURE(I)$ contains the set of items I_0 .

$E' \rightarrow \cdot E$
 $E \rightarrow \cdot E + T$
 $E \rightarrow \cdot T$
 $T \rightarrow \cdot T * F$
 $T \rightarrow \cdot F$
 $F \rightarrow \cdot (E)$
 $F \rightarrow \cdot id$

$CLOSURE(I)$

→ Two classes

1. Kernel items:

* Initial item, $S' \rightarrow \cdot S$ and all items where dots are not at left end.

2. Non Kernel items:

* Items with dots at left end except $S' \rightarrow \cdot S$

GOTO

→ The function is $GOTO(I, X)$

$I \rightarrow$ set of items

$X \rightarrow$ grammar symbol

→ Define the transitions in LR(0)

→ Specifies transition from state under input X .

$GOTO[I_i, A] = I_j$

maps a state i and non-terminal A to state j

$$I \Rightarrow \{ [E' \rightarrow E.], [E \rightarrow E. + T] \}$$

Goto(I, +) contains

[When a symbol
present before it, then
move it]

$$E \rightarrow E. + T$$

→ check any non-terminal
after.

$$T \rightarrow . T * F$$

$$T \rightarrow . F$$

$$F \rightarrow . (E)$$

$$F \rightarrow . id$$

Action:

→ Action four forms

* Shift

* Reduce

* Accept

* Error

Canonical Collection of LR(0) items

$$S \rightarrow AA$$

$$A \rightarrow aA/b$$

Augmented grammar $\Rightarrow$

$$S' \rightarrow S$$

$$S \rightarrow AA$$

$$A \rightarrow aA/b$$

→ we have to perform closure and ~~go~~ Goto to get the Canonical collection of LR(0).

→ Perform LR(0), give $\cdot$ to the production

$$S' \rightarrow \cdot S$$

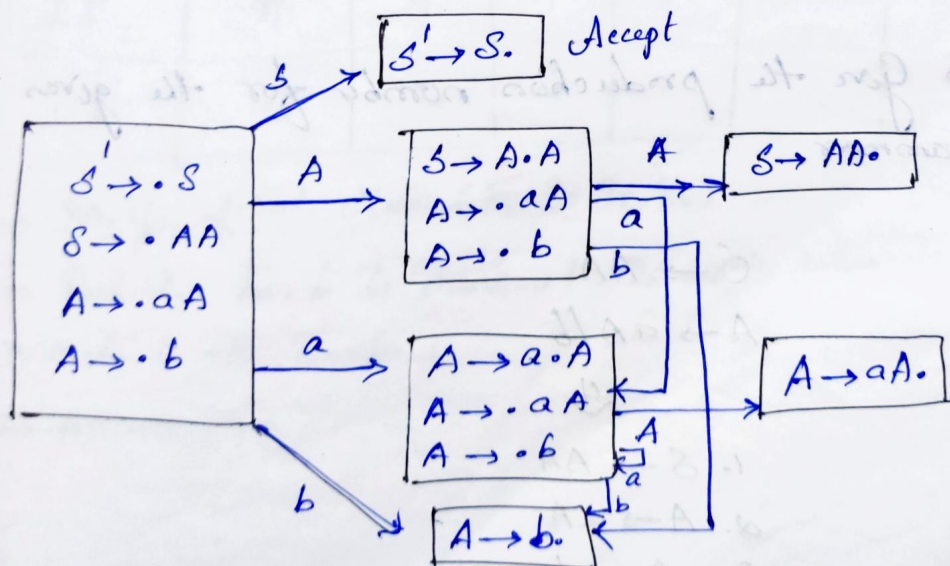
Find closure for the above

$$S' \rightarrow \cdot AA \quad [\text{check any grammar present for non-terminal } A]$$

$$A \rightarrow \cdot aA$$

$$A \rightarrow \cdot b$$

[After $\cdot$ if terminal value, we don't want to add any new production]



LR(0) Parsing Table

→ The parsing table is divided into two parts

(i) Action

(ii) GOTO

→ The code for the actions are

S_i means shift and stack state i

r_j means reduce by production number j

acc means accept

blank means error

→ The value of $GOTO[S, A]$ for terminal 'a' is found in ACTION field connected with the shift action on input a for state S.

→ The GOTO field gives $GOTO[S, A]$ for non-terminal A

→ Give the production number for the given grammar

$$S \rightarrow AA$$

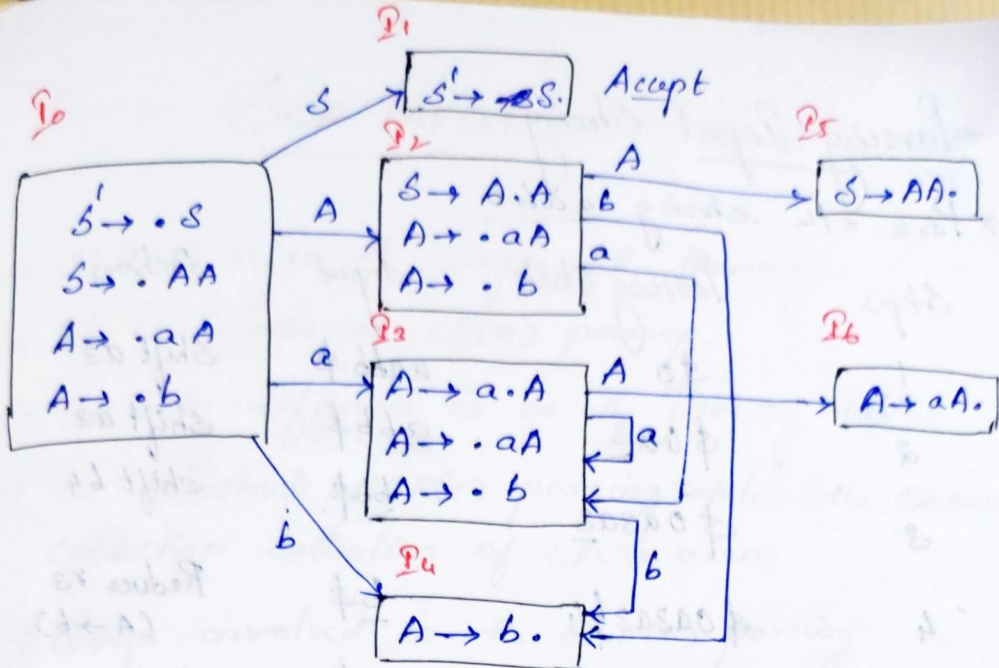
$$A \rightarrow aA / b$$

↓

1. $S \rightarrow AA$

2. $A \rightarrow aA$

3. $A \rightarrow b$



State	ACTION			GOTO	
	a	b	\$	S	A
I_0	S_3	S_4		1	2
I_1			acc		
I_2	S_3	S_4		5	
I_3	S_3	S_4		6	
I_4	r_3	r_2	r_2		
I_5	r_1	r_1	r_1		
I_6	r_2	r_2	r_2		

→ Identify final items (I_4, I_5, I_6)

→ Reduction has to be mentioned in acc. column accordingly to the number.

→ Fill the same

→ Parsing Input String

→ Parse the string aabbb

Steps	Parsing stack	Input	Action
1	\$0	<u>a</u> abb\$	Shift a_3
2	\$0a <u>3</u>	<u>a</u> bb\$	Shift a_3
3	\$0a3a <u>3</u>	<u>b</u> bb\$	Shift b_4
4	\$0a3a3 <u>b</u> 4	<u>b</u> \$	Reduce r_3 ($A \rightarrow b$)
5	\$0a3a3 <u>A</u> 6	<u>b</u> \$	Reduce r_2 ($A \rightarrow aA$)
6	\$0a3 <u>A</u> 6	<u>b</u> \$	Reduce r_2 ($A \rightarrow aA$)
7	\$0 <u>A</u> 2	<u>b</u> \$	Shift b_4
8	\$0A2 <u>b</u> 4	<u>\$</u>	Reduce r_3 ($A \rightarrow b$)
9	\$0A2A <u>5</u>	<u>\$</u>	Reduce r_1 ($S \rightarrow AA$)
10	\$0 <u>S</u> 1	<u>\$</u>	Accept

Parse Tree

